Root-Cause Diagnosis for MIMO Semantic Communications

Implementation report: data, models, settings and analyses

Purpose: this report documents how the DeepSC-MIMO root-cause diagnosis study was implemented: the data, the models, every preprocessing step, every setting and random seed, and the analyses behind each result in the paper "Root-Cause Diagnosis for MIMO Semantic Communications".

Source: all details are taken from the project code (the repository `deepsc-mimo-diagnosis`: the main pipeline at its root, the add-ons in `experiments/`, `stage_subset/`, `classical_tail/` and `classical_freeze_recovery/`) and from the stored result files. Where a quantity was not recorded by a run, this is stated.

Boxes used:

- Summary the main point of a section.
- In the code exact settings, files and functions.
- Design rationale why a choice was made.
- Example a worked example.
- Results measured numbers.
- Note limitations and implementation details to be aware of.

Lara Chehade · Supervisor: Dr. Hadi Sarieddeen · AUB VIP · October 2026

Contents

1 Overview of the study **2**

2 The map: what runs, in what order **2**

2.1 The main pipeline (`server/launch_pipeline.sh`) . . . 3

2.2 Add-on studies run after the main pipeline . . . 3

2.3 Main files of the repository . . . 3

3 Text data and vocabulary **4**

3.1 Cleaning a sentence (`preprocess_text.py`, from the original DeepSC code) . . . 4

3.2 The shared vocabulary . . . 4

3.3 Batches (`dataset.py`) . . . 5

4 The DeepSC network in code **5**

5 Training the two models C and D **5**

5.1 Why training ran in 8 chunks . . . 6

6 The MIMO channel in code **6**

6.1 From 16 real numbers to two antennas . . . 6

6.2 Random draws, reproducibly . . . 6

6.3 What `__call__` does, line by line . . . 6

7 Collecting the data: `collect_profiles.py` **7**

7.1 60 jobs: 12 conditions $\times$ 5 physical seeds . . . 7

7.2 Which sentences . . . 7

7.3 What each condition changes in the code . . . 8

7.4 The literal-corruption algorithm (`profiles.py`, `corrupt_sequence`) . . . 8

7.5 Getting R_3: free-running decoding . . . 8

7.6 What one saved batch file contains . . . 8

8 Localization: `analysis/first_divergence.py` **9**

9 The classical route, part 1: 972 features per sentence **9**

9.1 How each checkpoint is turned into a matrix . . . 9

9.2 The features computed on every checkpoint . . . 10

9.3 Where missing values come from . . . 10

10 The classical route, part 2: the three pipelines **11**

10.1 Folds and class balancing (`balanced_indices`) . . . 11

10.2 The three pipelines, step by step . . . 11
10.3 Step: median imputation . . . 11
10.4 Step: standardization . . . 12
10.5 Step: ANOVA selection (`SelectKBest(f_classif, k=64)`) . . . 12
10.6 Step: mRMR selection (`MRMRSelector`, written in the project) . . . 12
10.7 The classifiers . . . 13
10.8 Results of all 15 combinations . . . 13
10.9 Freezing the classical model . . . 14

11 The neural route: `analysis/neural_diagnostics.py` **14**
11.1 From a batch file to a training example . . . 14
11.2 Masked pooling: from a sequence to one vector . . . 15
11.3 The three single-checkpoint encoders . . . 15
11.4 The fusion model . . . 15
11.5 Training (same for every neural model) . . . 15
11.6 65 jobs and the freeze . . . 16

12 The arXiv test: `analysis/evaluate_holdout.py` **16**

13 Paper extraction: metrics, probabilities, windows **16**
13.1 `paper_metrics.py`: quality per sentence (CPU) . . . 16
13.2 `fusion_probs.py`: saving the diagnostic's probabilities (GPU) . . . 17

14 Stage subsets and Shapley values **17**
14.1 `stage_subset/ablation.py` . . . 17
14.2 `stage_attribution_analysis.py`: from confusion matrices to Shapley values . . . 18

15 The window follow-up scripts **18**

16 The correction experiment in code (`correction_v1`) **18**
16.1 The files . . . 19
16.2 What `run_fold.py` does, in order . . . 19
16.3 Resources and why . . . 19

17 Which features the classical models really use **20**
17.1 What follows directly from the code . . . 20

18 Glossary of the tools **21**

1 Overview of the study

Summary

DeepSC (a Transformer-based semantic communication system for text) is run over a 2×2 MIMO channel. Seven operating conditions degrade the output: nominal, low SNR, imperfect CSI, spatial correlation, literal corruption of the input text, transmitter–receiver model mismatch, and out-of-domain text. The study builds a **receiver-only diagnostic** that identifies the condition from four internal receiver signals (R_0 received waveform, R_1 equalized symbols, R_2 channel-decoder output, R_3 semantic-decoder state), without access to the channel, the SNR or the transmitted sentence.

Work carried out, in order:

1. **Testbed.** Two DeepSC models (C and D, identical except for the random seed) trained over the MIMO channel; seven conditions collected under five independent physical seeds (6,400 sentences per condition), with all four receiver stages recorded (Chapters 3–7).
2. **Localization.** Paired replays (same sentences, channel and noise) verify the first receiver stage altered by each physical and model impairment (Chapter 8).
3. **Diagnosis.** A classical route (972 engineered features per sentence; ANOVA, mRMR and elastic-net classifiers) and a neural route (CNN, TCN, convolutional Transformer, four-stage fusion), evaluated with leave-one-physical-seed-out cross-validation (Chapters 9–11).
4. **Held-out domain.** arXiv text tested once, after all model choices were frozen (Chapter 12).
5. **Analyses.** Per-sentence quality metrics, calibration, observation windows, channel conditioning, and a Shapley attribution over all 15 receiver-stage subsets (Chapters 13–15).
6. **Restoration.** Learned, cause-specific correction maps, routed by the diagnostic (Chapter 16).

Main results

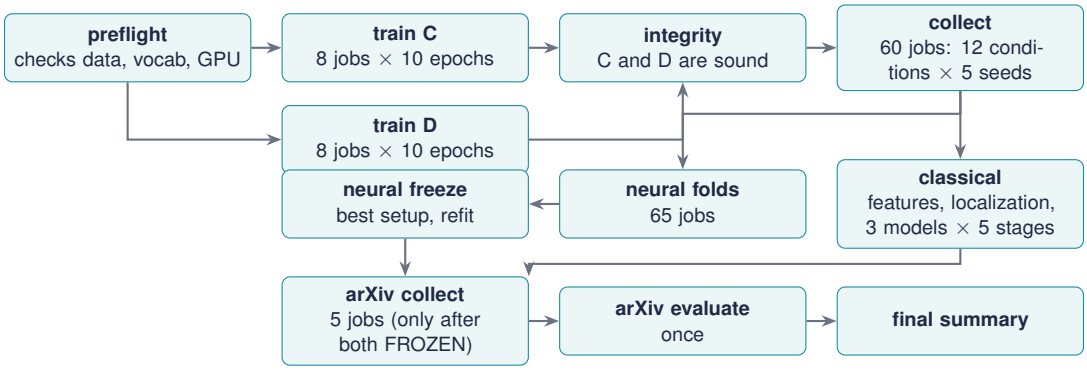
- Seven-way, single-sentence diagnosis: macro-F1 **0.70** for both fusion models (elastic net 0.704, neural fusion 0.696), against at most 0.62 for any single stage and 0.14 for chance; 0.79 balanced accuracy when 32 sentences of one channel realization are combined.
- Localization verified in all 400 paired batches: low SNR and spatial correlation at R_0, imperfect CSI at R_1, model mismatch at R_2.
- Stage attribution: channel impairments are best revealed at R_0, source impairments at R_3, imperfect CSI at R_2 (after its entry stage); R_0 and R_3 are the least redundant stages.
- Restoration: within the common receiver, routing by the diagnostic reaches 97% of the gain of routing by the true cause; a structured 2×2 map recovers 51%, 26% and 19% of the BLEU-1 loss from imperfect CSI at $\rho = 0.99, 0.95, 0.90$.

2 The map: what runs, in what order

Summary

Everything is one chain of cluster jobs. Each job only starts when the one it depends on succeeded, writes into its own folder, and the stages that produce results refuse to overwrite them. That's what makes every number traceable.

2.1 The main pipeline (`server/launch_pipeline.sh`)



2.2 Add-on studies run after the main pipeline

Folder	What it does	Chapter
`classical_tail`, `classical_freeze_recovery`	finish the slow fusion elastic-net fits that hit the time limit	10
`experiments/paper_extraction`	per-sentence probabilities, BLEU, calibration, window, condition number	13
`stage_subset`	the 15 checkpoint subsets with one fixed TCN	14
`experiments/window_analysis`	aggregation rules, genie Bayes test, cross-batch pooling	15
`experiments/correction`	the restoration maps and routing	16

2.3 Main files of the repository

File	Role
`preprocess_text.py`	original DeepSC text cleaning and vocabulary building
`dataset.py`	loads the pickled token lists, pads batches
`models/transceiver.py`	the DeepSC network
`channels.py`	the 2×2 MIMO channel, CSI, correlation, LMMSE
`utils.py`	loss, power normalization, SNR conversion, forward pass, training step
`main.py`	training with exact resume
`profiles.py`	the literal-corruption algorithm and profile settings
`collect_profiles.py`	runs a trained pair under one condition and saves R_0–R_3
`analysis/first_divergence.py`	paired localization check
`analysis/extract_sentence_features.py`	972 engineered features per sentence
`analysis/classical_diagnostics.py`	ANOVA, mRMR and elastic-net pipelines, leave-one-seed-out
`analysis/neural_diagnostics.py`	CNN, TCN, conv-Transformer and fusion classifiers
`analysis/evaluate_holdout.py`	the one-time arXiv test
`stage_subset/ablation.py`	15 checkpoint subsets with one TCN per checkpoint

In the code: safety rules used everywhere

- **Immutable outputs:** every script checks `if destination.exists(): raise FileExistsError`. A rerun can never silently overwrite a result.
- **Atomic writes:** files are written to a temporary name, then renamed. A job killed mid-write leaves no half-file.
- **Fingerprints:** each run stores SHA-256 hashes of its code (`code.sha256`) and of the checkpoints, vocabulary and datasets it used, and re-checks them before running.
- **FROZEN markers:** the arXiv jobs can't start until the classical and neural model choices are written to disk with a `FROZEN` file. This guarantees that arXiv did not influence any model choice.
- **Thread limits:** `OMP_NUM_THREADS` and `MKL_NUM_THREADS` are set to the number of requested CPUs, so jobs don't oversubscribe the node.

3 Text data and vocabulary

3.1 Cleaning a sentence (`preprocess_text.py`, from the original DeepSC code)

Each raw Europarl line goes through `normalize_string`:

1. **Unicode to ASCII:** accents are split off and dropped ("é" → "e").
2. **Remove XML/HTML tags** (`w3lib.remove_tags`).
3. **Space before** ! . ? so they become separate tokens.
4. **Keep only letters and** . ! ?: everything else (digits, commas, quotes) becomes a space.
5. **Collapse spaces, lower-case.**

Then `cutted_data` keeps sentences with **more than 4 and fewer than 30 words**. Each kept sentence is split on spaces and wrapped as `<START> w1 ... wn <END>`.

Example

"The Commission's proposal (COM 2004) is welcome!" becomes `<START> the commission s proposal com is welcome ! <END>`.

3.2 The shared vocabulary

One vocabulary of **90,746 tokens** is shared by Europarl, Hansard and arXiv, with fixed special IDs `<PAD>`=0, `<START>`=1, `<END>`=2, `<UNK>`=3. `main.py` refuses to run if the size or these IDs differ (`validate_vocabulary`).

Design rationale

A shared vocabulary means the same word has the same ID in every domain, so out-of-domain text isn't "out of vocabulary" by construction. Hansard and arXiv words exist in the vocabulary, but model C never *trained* on many of them (22.1% of arXiv tokens never appear in the Europarl training set). That's what makes OOD subtle.

3.3 Batches (`dataset.py`)

The data are pickled lists of token IDs. `collate_data` sorts a batch by length (longest first) and pads with 0 (<PAD>) to the longest sentence. Training batches are 128 sentences.

4 The DeepSC network in code

Block	What the code builds (`models/transceiver.py`)	Parameters
Semantic encoder	word embedding (90,746 × 128) multiplied by $\sqrt{128}$, plus sinusoidal positions; 4 layers of [8-head self-attention, add, LayerNorm; feed-forward 128→512→128 with ReLU, add, LayerNorm]; dropout 0.1	12.4 M
Channel encoder	Linear 128→256, ReLU, Linear 256→16	37 k
Power normalization	divide the whole batch tensor by $\max(\text{rms}, 1)$	–
MIMO channel	`channels.py` (Chapter 6)	–
Channel decoder	x_1 = Linear 16→128; ReLU; Linear 128→512; ReLU; Linear 512→128 = x_5; output LayerNorm$(x_1 + x_5)$	134 k
Semantic decoder	embedding + positions; 4 layers of [masked self-attention; attention over the channel-decoder output; feed-forward], each with add and LayerNorm	12.7 M
Output layer	Linear 128→90,746 (scores for every word)	11.7 M
Total		**37.0 M**

Almost all parameters are in the two embedding tables and the output layer, because the vocabulary is large. The parts that touch the channel are tiny (37 k and 134 k).

In the code: details that matter later

- **Power normalization** (`PowerNormalize` in `utils.py`): rms = $\sqrt{\text{mean}(x^2)}$ over *all* values of the batch, and the batch is divided by $\max(\text{rms}, 1)$. So it only ever scales *down*, and one sentence's symbols depend slightly on the rest of its batch.
- **Masks:** the source mask hides padding; the decoder's look-ahead mask stops each position from seeing future words.
- **Initialization:** every weight matrix is Xavier-uniform (`initNetParams`).
- **R_2 is the channel-decoder output**, 128 numbers per word. It is the "memory" the semantic decoder attends to.

5 Training the two models C and D

In the code: `main.py` settings used

- **Optimizer:** Adam, learning rate 10^{-4}, $\beta = (0.9, 0.98)$, $\epsilon = 10^{-8}$, weight decay 5×10^{-4}. No learning-rate scheduler (the original DeepSC code has none).
- **Epochs and batches:** 80 epochs, batch 128, no shuffling (the original order).
- **Noise during training ("legacy epoch" mode):** one SNR per epoch, drawn uniformly between the noise levels of 5 dB and 10 dB. All batches of that epoch use it.
- **Channel during training:** a fresh 2×2 Rayleigh **H** per batch, perfect CSI, LMMSE. Each epoch uses its own reproducible "realization set" (index = epoch number).

- **Loss:** cross-entropy of the next word (teacher forcing: the decoder sees the true previous words), with padding positions multiplied by zero. "Legacy" normalization: the sum is divided by *all* positions, padding included, as in the original code.
- **Validation:** fixed noise standard deviation 0.1 per real component (about 17 dB) and a fixed realization set (1,000,000), so the validation loss is comparable across epochs.
- **Seeds:** model C uses 42 (global and channel), model D uses 43. That's the only difference between them.

5.1 Why training ran in 8 chunks

A cluster job has a time limit (5 h 50 min here), so each model was trained as 8 dependent jobs of 10 epochs. Each chunk saves a full checkpoint: model weights, optimizer state, the epoch, **every random-number-generator state** (Python, NumPy, PyTorch CPU and GPU), and the next channel realization set. The next chunk resumes from it and refuses to continue if the epoch and the realization set disagree. So chunked training is *exactly* the same as one 80-epoch run.

Design rationale

Without saving the random states, a resumed run would draw different channels and noise than an uninterrupted one, and C would no longer be reproducible. `last.pth` at epoch 80 is the canonical checkpoint; `best.pth` (lowest validation loss) is only kept as a sensitivity check.

6 The MIMO channel in code

6.1 From 16 real numbers to two antennas

`real_interleaved_to_complex` reads the 16 real numbers of a word as pairs (real, imaginary), giving 8 complex symbols, then groups consecutive symbols by 2 (one per antenna). So a sentence of L tokens becomes $4L$ channel uses, each a 2-vector. Then every symbol is multiplied by $1/\sqrt{2}$ (total power over the two antennas = single-antenna power).

6.2 Random draws, reproducibly

In the code: `MIMOChannel._generator`

Each kind of randomness has its **own** generator:

$$\text{seed}_{\text{stream}} = \text{physical seed} + 1{,}000{,}003 \times \text{offset} + 10{,}000{,}019 \times \text{realization set},$$

with offset 0 for the channel, 1 for the noise, 2 for the CSI error. Because the streams are separate, switching on imperfect CSI draws extra CSI numbers *without shifting* the channel or noise draws. That's why nominal and impaired runs share exactly the same $\mathbf{H}$ and noise: the basis of the paired localization.

6.3 What `__call__` does, line by line

1. **Channel:** $\mathbf{H}_w$ with $\mathcal{CN}(0,1)$ entries (real and imaginary parts $\mathcal{N}(0,1/2)$), shape $1 \times 1 \times 2 \times 2$: one per batch.
2. **Correlation** (if on): $\mathbf{H} = \mathbf{S}\mathbf{H}_w\mathbf{S}$ with $\mathbf{S} = \mathbf{R}^{1/2}$ computed from the eigen-decomposition of

$\mathbf{R} = \begin{bmatrix} 1 & \rho_s \\ \rho_s & 1 \end{bmatrix}$. The unit diagonal keeps the average power at 1.

3. **CSI estimate** (if on): $\hat{\mathbf{H}} = \rho\mathbf{H} + \sqrt{1-\rho^2}\,\tilde{\mathbf{H}}$, $\tilde{\mathbf{H}}$ from the CSI stream. Otherwise $\hat{\mathbf{H}} = \mathbf{H}$.
4. **Faded signal: Hx** for every channel use.
5. **Noise:** complex Gaussian with standard deviation $\sigma = 1/\sqrt{2 \cdot 10^{\mathrm{SNR}/10}}$ per real part (`SNR_to_noise`), so complex variance $2\sigma^2$.
6. **LMMSE:** with the assumed symbol covariance $\mathbf{R}_x = \mathbf{I}$,

$$\mathbf{G} = \mathbf{R}_x\hat{\mathbf{H}}^{\mathsf{H}}\left(\hat{\mathbf{H}}\mathbf{R}_x\hat{\mathbf{H}}^{\mathsf{H}} + 2\sigma^2\mathbf{I}\right)^{-1},$$

   computed with `torch.linalg.solve` (solving a linear system is more accurate than inverting a matrix).
7. **Back to 16 real numbers per word** (`complex_to_real_interleaved`): this is R_1, the channel-decoder input.

Note

The receiver assumes $\mathbf{R}_x = \mathbf{I}$ (unit-power, uncorrelated streams). The code also records the *actual* covariance of the transmitted symbols, but only in the trace; the equalizer never uses it. Whether DeepSC's learned symbols are really white is an open question (it matters for the CSI correction, Section V-F of the paper).

7 Collecting the data: `collect_profiles.py`

7.1 60 jobs: 12 conditions × 5 physical seeds

`config/development_profiles.csv` lists 12 rows: normal, low SNR, the 19 dB control, imperfect CSI at 0.99/0.95/0.90, correlation at 0.3/0.6/0.9, literal (0.20), mismatch, Hansard. Job number t (0–59) runs row $\lfloor t/5 \rfloor$ with seed number t mod 5, i.e. physical seed 3101–3105.

7.2 Which sentences

In the code: sentence selection

1. Load the domain's test pickle (Europarl, Hansard or arXiv).
2. Shuffle all indices with `random.Random(20260823 + offset)`, offset 0 / 100 / 200 for Europarl / Hansard / arXiv.
3. Seed index s takes the block of 1,280 indices starting at 1,280 s: **the five physical seeds use five disjoint sets of sentences**.
4. Batches: 10 consecutive groups of 128, in that shuffled order (not sorted by length).

Design rationale

Same selection seed for every condition, so seed 3101 batch 0 contains the same 128 sentences under normal, low SNR, CSI, and so on: the pairing. Disjoint blocks across seeds, so leave-one-seed-out tests on sentences *and* channels the classifier never saw.

7.3 What each condition changes in the code

Condition	Setting
normal	C→C, `SNR_to_noise(10)`, `profile="baseline"`
low SNR	same, `SNR_to_noise(1)`
control	same, `SNR_to_noise(19)` (not a class)
imperfect CSI	`profile="imperfect_csi"`, `csi_rho` $\in \{0.99, 0.95, 0.90\}$
correlation	`profile="spatial_correlation"`, `spatial_rho` $\in \{0.3, 0.6, 0.9\}$
literal	sentences corrupted before encoding (below)
mismatch	`tx_model` = C, `rx_model` = D
Hansard	domain `ukhansard` test file, C→C

7.4 The literal-corruption algorithm (`profiles.py`, `corrupt_sequence`)

For one sentence:

1. Count the editable tokens (everything except PAD, START, END, UNK). Number of edits $= \max(1, \text{round}(0.20 \times \text{count}))$.
2. For each edit, pick an operation with probabilities **deletion 35%, typo 30%, repeat 20%, grammar swap 15%**. If it can't apply, try typo, then deletion, then repeat.
   - **Deletion:** remove a random token, only if the sentence stays longer than 7 tokens.
   - **Repeat:** copy a token (or a neighbour) right after itself, only if shorter than 31.
   - **Grammar swap:** replace a word by its pair from a fixed list (is/are, was/were, a/the, in/on, to/for, he/they, ...).
   - **Typo:** in a word of 3+ letters, delete, swap, substitute or insert one letter. The new word's ID is looked up; if it isn't in the vocabulary it becomes <UNK>.
3. Keep START and END in place and the length within 31.

The random generator is seeded per sentence ($7301 \times 1{,}000{,}003 +$ sample id), so the same sentence is always corrupted the same way. The *clean* sentence stays the reference for BLEU.

7.5 Getting R_3: free-running decoding

In the code: `greedy_hidden_states`

Start with <START>. At each step (at most 30), run the decoder on everything generated so far, take the **last position of the final decoder layer** (128 numbers, before the output layer): that's one R_3 vector. Pick the highest-scoring word and append it. A sentence stops after predicting <END>; the state that predicted <END> is kept. Finished sentences get zero states afterwards, and a mask records each sentence's length.

Design rationale

Free-running, not teacher-forced: at the receiver the true previous words aren't known, so R_3 must come from the decoder's own guesses. That's also why R_3 "thinks in language": each state depends on the words the decoder has already written.

7.6 What one saved batch file contains

`batch_000.pt` ... `batch_009.pt`, each with: metadata (condition, label, severity, SNR, seed, batch index, sample IDs); the source and clean reference tokens and their lengths; the corruption log

(literal); R_0 (received, $128 \times 4L \times 2$ complex); R_1 ($128 \times L \times 16$); R_2 ($128 \times L \times 128$); R_3 ($128 \times$ steps $\times$ 128), the generated words, lengths and mask; and the audit-only physics: true $\mathbf{H}$, the underlying $\mathbf{H}_w$, $\hat{\mathbf{H}}$, the faded signal, the noise and the standardized noise. Plus hashes of the configuration and of both checkpoints.

Note

The physics fields ($\mathbf{H}$, $\hat{\mathbf{H}}$, noise) are saved **for auditing only**. No diagnostic ever reads them: the feature extractor and the neural datasets read only R_0–R_3 and the lengths.

8 Localization: `analysis/first_divergence.py`

1. Load all normal batches and index them by (seed, batch, sample IDs).
2. For each low-SNR, CSI, correlation and mismatch batch, find its normal partner (it must exist, or the script stops).
3. Compare, in order, the source tokens, R_0, R_1, R_2, R_3 with **exact equality** (`torch.allclose(..., rtol=0, atol=0)`). The first unequal one is the entry stage.
4. Stop with an error if it differs from the expected map (low SNR R_0, correlation R_0, CSI R_1, mismatch R_2), and check extra invariants: low SNR keeps $\mathbf{H}$ and the standardized noise (to 2×10^{-6}); CSI keeps $\mathbf{H}$, the noise and R_0; correlation keeps $\mathbf{H}_w$.

Results

400 paired batches checked, all as expected: low SNR R_0 in 50, correlation R_0 in 150, CSI R_1 in 150, mismatch R_2 in 50.

Design rationale

Exact equality, not "close enough", because the replays use the same random numbers: anything upstream of the change must be bit-for-bit identical. A tolerance would hide a pairing bug.

9 The classical route, part 1: 972 features per sentence

Summary

The classical diagnostic does not use the raw signals. `analysis/extract_sentence_features.py` turns each sentence's four checkpoints into **972 summary numbers** ("features"), one row per sentence in a big table (`development_features.csv`). The classifiers only ever see that table.

9.1 How each checkpoint is turned into a matrix

For one sentence, each checkpoint is a matrix with one row per time step and D columns:

Checkpoint	Rows (time steps)	Columns D	What a column is
R_0	$4\times$ source length (channel uses)	4	Re/Im of antenna 1, Re/Im of antenna 2
R_1	source length (tokens)	16	the 16 equalized numbers
R_2	source length	128	channel-decoder outputs
R_3	generated length	128	decoder hidden state

Only the real rows are used: padding is cut off using the stored lengths ("padding-aware").

9.2 The features computed on every checkpoint

With x = all entries of the matrix flattened, and M the matrix:

Feature name (suffix)	Definition	Count
Distribution of all values		
`mean, std, min, max`	of all entries (std with $1/N$)	4
`rms, energy`	$\sqrt{\overline{x^2}}$ and $\sum x^2$	2
`p10, p25, p50, p75, p90`	percentiles	5
`abs_mean, zero_fraction`	$\overline{\lvert x\rvert}$; share of exact zeros	2
`gini_energy`	Gini coefficient of x^2: 0 = energy spread evenly, near 1 = concentrated in a few entries	1
`skew, kurtosis`	$\overline{z^3}$ and $\overline{z^4}$ with $z = (x - \bar{x})/\text{std}$ (kurtosis 3 for a Gaussian)	2
Per column d		
`dim005_mean, _std, _rms`	mean, std, rms of column d over time	$3D$
Shape of the matrix (columns centred first; s_i = singular values, $p_i = s_i^2 / \sum s_j^2$)		
`spectral_entropy`	$-\sum p_i \log p_i$	1
`effective_rank`	e^{entropy}: "how many directions" the signal really uses	1
`top1_concentration, top3_...`	p_1; $p_1 + p_2 + p_3$	2
`singular_max, singular_min, condition`	s_1, $s_{\min}$, $s_1/s_{\min}$	3
`covariance_abs_mean, _offdiag_abs_mean`	mean $\lvert$entry$\rvert$ of the $D \times D$ covariance; same for off-diagonal	2
`row_norm_mean, row_norm_std`	length of each time step's vector: mean, std	2
Time structure, for lags $\ell = 1, 2, 5$		
`lag1_cosine_mean`	mean cosine similarity between step t and step $t + \ell$	3
`lag1_delta_rms`	rms of the difference between step t and $t + \ell$	3
	Total per checkpoint	$\mathbf{33 + 3D}$

Only on R_0 (`r0_mimo_features`), from the 2×2 sample covariance of the received vectors: `rx0_power`, `rx1_power` (power per receive antenna), `antenna_corr_abs`, `_real`, `_imag` (normalized correlation between the two antennas), `stream_singular_ratio` (ratio of the two singular values of the received samples: how "unbalanced" the two streams are).

Between checkpoints (`T__`): for each pair $R_0 \to R_1$, $R_1 \to R_2$, $R_2 \to R_3$, the ratio of rms values and its logarithm.

Results

Group	Count	How
R_0	51	$33 + 3 \cdot 4 = 45$, plus 6 MIMO features
R_1	81	$33 + 3 \cdot 16$
R_2	417	$33 + 3 \cdot 128$
R_3	417	$33 + 3 \cdot 128$
T	6	3 pairs $\times$ (ratio, log-ratio)
Fusion	**972**	all of the above

9.3 Where missing values come from

A feature can't always be computed. In the code:

- **Lag features** need more than ℓ steps. R_0 (at least 28 channel uses), R_1 and R_2 (at least 7 tokens, since sentences have 5+ words plus START and END) always have them. R_3 **often doesn't**: if the decoder stops after a few words, `lag5_...` (or even `lag1_...`) is missing.
- **Matrix-shape features** need at least 2 steps: missing for an R_3 of length 1.

- **Skew and kurtosis** need a nonzero standard deviation.

A missing feature is left out of that sentence's row; when the table is read back, the empty cell becomes `NaN` ("not a number"). How often each feature is missing was not tabulated by the run.

10 The classical route, part 2: the three pipelines

10.1 Folds and class balancing (`balanced_indices`)

For held-out seed s: training rows = the other 4 seeds; test rows = seed s. Then both sides are **balanced**:

1. Find the smallest class size $n_{\min}$ (5,120 sentences per class in training, 1,280 in test).
2. For CSI and correlation, take $n_{\min}/3$ from each severity, so the three severities are equally represented.
3. For other classes, take $n_{\min}$ at random.

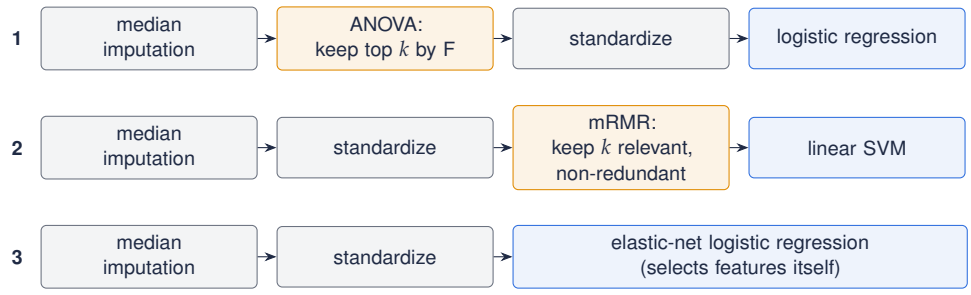
Seeds: `4401` + `s` for training, `4401` + `100000` + `s` for test. Result: **35,840 training and 8,960 test sentences** per fold, 7 classes × equal counts. Every model and every checkpoint uses exactly the same rows.

Design rationale

CSI and correlation have 3 severities × 6,400 sentences each, three times more than the other classes. Without balancing, a classifier would learn "guess CSI" and look better than it is. Balancing severities stops it from learning only the easy severe cases.

10.2 The three pipelines, step by step

Each is a scikit-learn `Pipeline`: every step is fitted on the **training fold only**, then applied unchanged to the test fold.



$k = \min(64, \text{number of features})$. So on R_0, which has only 51 features, ANOVA and mRMR keep **all 51**: selection does nothing there.

10.3 Step: median imputation

In the code: `SimpleImputer(strategy="median")`

During fitting, for each feature (column) it stores the **median of the training-fold values**, ignoring NaNs. Afterwards, every NaN in that column, in training or test, is replaced by that stored median.

Example

Feature `R3__lag5_cosine_mean` in five training sentences: 0.42, 0.51, NaN, 0.47, 0.39. The median of the four known values is $(0.42 + 0.47)/2 = 0.445$. The NaN becomes 0.445, and so does any test sentence with a missing value there.

Design rationale

The classifiers can't handle NaNs. The **median** instead of the mean, because a few extreme sentences (very short outputs, garbled mismatch outputs) would drag the mean. The median comes from **training only**, so the test fold never influences preprocessing (no leakage).

Note

Imputation hides the fact that a value was missing, and "missing" is itself informative here: a short R_3 means the decoder stopped early. A missing-value indicator column would keep that information; the pipeline doesn't add one.

10.4 Step: standardization

In the code: `StandardScaler()`

For each feature, store the training mean μ and standard deviation σ, then replace every value by $z = (x - \mu)/\sigma$. After this, each feature has mean 0 and standard deviation 1 on the training fold.

Design rationale

Features live on wildly different scales: `R2__energy` can be in the thousands, a cosine is between -1 and 1. Regularized models (all three here) penalize large weights, so without scaling, a feature's *units* would decide how much it's penalized. Scaling also lets the saga solver converge. It doesn't change ANOVA or mRMR rankings (both are unaffected by shifting and rescaling a feature).

10.5 Step: ANOVA selection (`SelectKBest(f_classif, k=64)`)

For each feature on its own, the **F-statistic** compares how far apart the class means are with how spread out each class is:

$$F = \frac{\frac{1}{K-1}\sum_{c=1}^{K} n_c(\bar{x}_c - \bar{x})^2}{\frac{1}{N-K}\sum_{c=1}^{K}\sum_{i \in c}(x_i - \bar{x}_c)^2}, \qquad K = 7 \text{ classes.}$$

Large F: the classes sit at clearly different levels of this feature. The 64 features with the largest F are kept.

10.6 Step: mRMR selection (`MRMRSelector`, written in the project)

Minimum redundancy, maximum relevance, greedy:

1. Relevance of each feature = its F-statistic divided by the largest F (so between 0 and 1).

2. Start with the most relevant feature.
3. Repeatedly add the feature with the highest **relevance − mean |correlation| with the features already chosen**.
4. Stop at 64.

Design rationale

ANOVA's weakness: it scores features one at a time, so its top 64 can be 64 near-copies of the same information. mRMR penalizes a candidate for repeating what's already chosen (Peng et al. 2005 use mutual information; this project uses the F-statistic for relevance and the absolute Pearson correlation for redundancy, a common cheaper variant).

10.7 The classifiers

In the code: exact settings

- **Logistic regression** (after ANOVA): `LogisticRegression(max_iter=2000, class_weight="balanced")`, default L2 penalty with $C = 1$, multinomial (softmax) over 7 classes: $p(c \mid \mathbf{z}) = \dfrac{e^{\mathbf{w}_c^\top \mathbf{z} + b_c}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{z} + b_{c'}}}$.
- **Linear SVM** (after mRMR): `LinearSVC(class_weight="balanced", max_iter=5000)`, $C = 1$, squared hinge loss, one-vs-rest (one boundary per class).
- **Elastic net**: `LogisticRegression(penalty="elasticnet", solver="saga", l1_ratio=0.5, max_iter=3000, class_weight="balanced")`. It minimizes

$$\tfrac{1-\rho}{2}\|\mathbf{W}\|_2^2 + \rho\|\mathbf{W}\|_1 + C\sum_i -\log p(y_i \mid \mathbf{z}_i), \qquad \rho = 0.5,\ C = 1.$$

  The $\|\mathbf{W}\|_1$ part pushes many weights to **exactly zero**, so the model selects its own features; the $\|\mathbf{W}\|_2^2$ part keeps groups of correlated features together instead of picking one at random (Zou & Hastie 2005).

`class_weight="balanced"` weights each class by $N/(K\,n_c)$; with already balanced folds, all weights are 1.

10.8 Results of all 15 combinations

Macro-F1 (mean ± std, 5 folds)	ANOVA + LR	mRMR + SVM	Elastic net
R_0	0.309 ± 0.026	0.286 ± 0.023	0.310 ± 0.027
R_1	0.378 ± 0.016	0.346 ± 0.020	0.380 ± 0.015
R_2	0.524 ± 0.017	0.521 ± 0.011	0.588 ± 0.026
R_3	0.340 ± 0.009	0.468 ± 0.016	0.590 ± 0.020
Fusion (972)	0.374 ± 0.015	0.572 ± 0.035	$\mathbf{0.704 \pm 0.050}$

What this table teaches

- **ANOVA collapses on R_3 (0.34) and on fusion (0.37)**, where there are hundreds of features. Its top 64, scored one at a time, are likely dominated by near-duplicates (for example many R_2/R_3 dimensions that all separate mismatch), so other causes lose their features. The feature job will show whether this is the reason.

- **mRMR fixes much of that** (0.47 on R_3, 0.57 on fusion) by penalizing redundancy.
- **Elastic net wins everywhere**: it sees all features and lets the L1 penalty choose, jointly rather than one at a time.
- On R_0 and R_1 the three methods tie, because there are few features and ANOVA/mRMR keep almost all of them.

A sanity check hidden in the data

On R_0, the elastic net's predictions for nominal and mismatch sentences of fold 3101 are *identical*: both rows of the confusion matrix read $[203, 140, 235, 272, 125, 34, 271]$. Mismatch only changes the receiver from R_2 on, so its R_0 *is* the nominal R_0, and the classifier can't do anything but treat them the same. The localization result, visible directly in a classifier output.

10.9 Freezing the classical model

The model with the best mean macro-F1 (fusion elastic net, 0.704) is refitted on all five seeds (balanced, $7 \times 6{,}400 = 44{,}800$ sentences) and saved as `frozen_classical.joblib`, with a `FROZEN` marker. Only then may arXiv be touched.

Note: the "recovery" folders

The fusion elastic net is slow (972 features, saga solver, 3,000 iterations). Some folds and the final refit hit the 5 h 50 min limit. Two small add-ons finished them without changing anything: `classical_tail` re-ran only the missing fusion folds (seeds 3103–3105) as one job per model, with the same seeds and sampling; `classical_freeze_recovery` ran the final refit as two "warm-start" chunks of 1,500 iterations (the second continues from the first's weights), and first re-checked that the selected model was still fusion elastic net. Their code fingerprints are stored in `provenance/hashes/` and match the repository.

11 The neural route: `analysis/neural_diagnostics.py`

Summary

The neural diagnostics read the checkpoints directly, as sequences, and learn their own features. One sentence = one variable-length sequence per checkpoint.

11.1 From a batch file to a training example

- `scan_records` lists every sentence of every development batch file with its label and seed. It refuses to start if an arXiv file is present.
- `stage_tensor` cuts out one sentence: R_0 the first $4L$ channel uses (complex $\to$ 4 real numbers), R_1/R_2 the first L tokens, R_3 the generated length. No hand-made features.
- `collate` pads the sentences of a mini-batch with zeros to the longest one and builds a mask (True = real step).
- Classes are balanced exactly as in the classical route (same rules, same counts: 35,840 / 8,960), with seeds `5501 + s` and `5501 + 100000 + s`.

Note

- **No input normalization.** The neural models get the raw values (unlike the classical route's standardization). The networks adapt, but checkpoints with very different scales are fed as is.
- **Padding is only masked in the pooling.** The convolutions run over the zero-padded positions too, so the last real steps of a short sentence "see" some zeros, and the TCN's batch normalization statistics include padded positions. This is a common, mild simplification; it's the same for every class, so it can't create a class-specific shortcut by itself, but sentence length does vary by class (e.g. literal corruption).

11.2 Masked pooling: from a sequence to one vector

Every encoder ends with `masked_pool`: over the real (unmasked) steps, take the **mean** and the **maximum** of each channel, and concatenate. A width-64 encoder gives a 128-number summary, whatever the sentence length.

11.3 The three single-checkpoint encoders

Encoder	Layers	Params (R_0/R_1/R_2, R_3)
CNN	Conv1d (kernel 5) $\rightarrow$ ReLU $\rightarrow$ Conv1d (kernel 3) $\rightarrow$ ReLU, width 64; masked mean+max pool; linear layer to 7 classes	15k / 18k / 54k
TCN	1×1 conv to width 64, then 4 residual blocks [dilated conv (kernel 3, dilation 1, 2, 4, 8) $\rightarrow$ batch norm, added back, ReLU]; pool; linear	51k / 52k / 59k
Conv-Transformer	Conv1d (kernel 3) $\rightarrow$ ReLU, sinusoidal positions, 2 Transformer layers (4 heads, feed-forward 128, dropout 0.1, padding mask); pool; linear	69k / 71k / 92k

Example: receptive field

A TCN block with dilation 4 compares a step with the steps 4 before and 4 after. Stacking dilations 1, 2, 4, 8 lets the top layer see $1 + 2\,(1 + 2 + 4 + 8) = 31$ consecutive steps, a whole sentence, with only 4 layers.

11.4 The fusion model

In the code: `FusionClassifier`

One CNN encoder per checkpoint, width 32 (each gives a 64-number summary), concatenated ($4 \times 64 = 256$), then Linear 256$\rightarrow$128 $\rightarrow$ ReLU $\rightarrow$ Dropout 0.1 $\rightarrow$ Linear 128$\rightarrow$7. About 90k parameters.

11.5 Training (same for every neural model)

In the code: `train_model`

AdamW, learning rate 3×10^{-4}, weight decay 10^{-4}; 12 epochs; mini-batches of 64, shuffled with a fixed generator (seed 5501); cross-entropy loss. Model initialization seeded with `5501 + held-out seed`. No early stopping, no validation split: 12 epochs, then test once.

Design rationale

No early stopping on the test fold means the test fold never steers training. Fixed seeds mean any fold can be rerun bit-for-bit (the paper-extraction job did exactly that, and matched within 0.008).

11.6 65 jobs and the freeze

3 encoders × 4 checkpoints + fusion = 13 setups, × 5 folds = 65 jobs. `freeze_mode` averages each setup's macro-F1 over its 5 folds and picks the best:

Macro-F1	R_0	R_1	R_2	R_3
CNN	0.308	0.412	0.589	0.580
TCN	**0.343**	0.414	0.596	0.582
Conv-Transformer	0.332	**0.436**	**0.617**	**0.602**
Fusion (4 CNNs)		**0.696**		

Fusion wins; it is retrained once on all five seeds' balanced data and saved with a `FROZEN` marker.

Note

The single-checkpoint rows of the paper's Table 2 use the best encoder per checkpoint, which mixes architectures (TCN on R_0, conv-Transformer elsewhere). That's why the stage-subset study (Chapter 14) re-did everything with one TCN for all.

12 The arXiv test: `analysis/evaluate_holdout.py`

Runs **once**, after both FROZEN markers exist:

1. Collect arXiv with C→C at 10 dB (an array of 5 jobs), labelled `HOLDOUT`.
2. Extract the same 972 features (`--include-holdout`), feed them to the frozen classical pipeline, count how many are called "ood".
3. Load the frozen fusion network, predict every arXiv sentence (labels mapped to "ood" for scoring).

Results

3,840 arXiv sentences. Classical: 33.4% "ood", 63.6% "literal". Neural fusion: 53.0% "ood".

13 Paper extraction: metrics, probabilities, windows

13.1 `paper_metrics.py`: quality per sentence (CPU)

In the code: how each quality number is computed

- **Reference:** the *clean* source tokens, stopping at `<END>`, with PAD/START/END/UNK removed. **Hypothesis:** the generated words up to the stored length, same cleaning.
- **BLEU-1:** NLTK `sentence_bleu` with weights (1): clipped unigram precision times the brevity penalty $\min(1, e^{1-r/h})$ (r, h = reference and hypothesis lengths).

- **BLEU-4:** weights $(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$: geometric mean of 1- to 4-gram precisions times the brevity penalty, with `SmoothingFunction().method1` (a zero count is replaced by 0.1, so one missing 4-gram doesn't zero the score).
- **Token accuracy:** share of reference positions i where the output has the same word at position i. A deletion shifts every later word, which is why it understates literal-corruption quality.
- **UNK rate** of the source, and **unseen rate**: share of source tokens that never appear in Europarl's training pickle (what C actually learned).
- **Per batch:** condition number $s_{\max}/s_{\min}$ of the true $\mathbf{H}$, and the CSI relative error $\|\hat{\mathbf{H}} - \mathbf{H}\|/\|\mathbf{H}\|$ (for analysis only).

13.2 `fusion_probs.py`: saving the diagnostic's probabilities (GPU)

The original neural folds saved only scores. To analyse errors, each fusion fold was **re-trained with the identical seeds and settings**, and the softmax probabilities were saved for *every* sentence of the held-out seed (not only the balanced subset), with a flag marking the original balanced test rows.

Results

Reproduction check: on the flagged rows, balanced accuracy is within 0.008 of the original folds.

Then the analysis mode (CPU, numbers only):

- **Per-severity recall** for CSI and correlation.
- **Expected calibration error** with 15 equal-width bins of the top probability: $\text{ECE} = \sum_b \frac{n_b}{N} |\text{accuracy}_b - \text{confidence}_b|$.
- **Observation window:** within one batch file (one channel draw), cut the sentences into consecutive, non-overlapping groups of k; average their 7 probabilities; predict the top class. $k \in \{1, 2, 4, 8, 16, 32, 64, 128\}$ was fixed in the code before any result.
- **Condition-number terciles**: recall split by the batch's $\kappa(\mathbf{H})$.

14 Stage subsets and Shapley values

14.1 `stage_subset/ablation.py`

In the code

- **Model** `SubsetTCN`: one unchanged TCN encoder per checkpoint in the subset, embeddings concatenated, one linear layer to 7 classes. All four: 221k parameters; without R_1: 169k (23% fewer).
- **Jobs:** 11 multi-checkpoint subsets $\times$ 5 folds = 55 GPU jobs, plus one pilot job (all four checkpoints, seed 3101) run first to measure the time. The 4 single-checkpoint subsets reuse the TCN folds of Chapter 11, which used the same protocol.
- **Same samples everywhere:** each job stores a SHA-256 digest of its exact train and test sentence lists. The summary refuses to run if two subsets of the same fold used different sentences, or if the training settings differ.

14.2 `stage_attribution_analysis.py`: from confusion matrices to Shapley values

All numbers are computed from the saved confusion matrices, fold by fold:

1. F1 of class k: $\dfrac{2\,\mathrm{TP}}{\text{predicted as } k + \text{truly } k}$; macro-F1 = mean over the 7 classes.
2. Value of a subset $v(\mathcal{S})$ = that F1; $v(\varnothing) = 1/7$.
3. Shapley value of each checkpoint and interaction index of each pair, with the standard weights $|\mathcal{S}|!\,(3-|\mathcal{S}|)!/4!$ and $|\mathcal{S}|!\,(2-|\mathcal{S}|)!/3!$.
4. Mean $\pm$ std over the 5 folds; consistency counts ("improves in x of 40 subset–fold pairs").
5. **Kendall's W:** each fold ranks the 15 subsets by macro-F1; $W = \frac{12\,S}{m^2(n^3-n)}$ with $m = 5$ folds, $n = 15$ subsets and S the spread of the summed ranks. $W = 1$ means every fold ranks the subsets the same; the measured value is 0.89.

15 The window follow-up scripts

All read only the saved fusion probabilities (`fusion_probs_seed*.csv`).

Script	What it does
`agg_sweep.py`	Combines k sentences with the rules of Kittler et al. (1998): sum (average), product (average of log-probabilities, with probability floors 10^{-12} to 10^{-2}), median, majority vote (ties broken by mean probability), 20%-trimmed mean. The rule is chosen **nested**: for each held-out fold, the rule best on the other four folds.
`bias_cal.py`	Same, but tunes per-class decision offsets, nested.
`bayes_corr.py`	The "genie": draws $\mathbf{H}$ (400,000 per class) under nominal and under correlation, and decides with the exact likelihood ratio. $\mathrm{vec}(\mathbf{H})$ is complex Gaussian with covariance $\mathbf{I}$ (nominal) or $\mathbf{R} \otimes \mathbf{R}$ (correlation); the three severities are an equal mixture.
`bayes_m.py`	The same genie, seeing $m = 1, 2, 3, 5, 8$ independent channel draws (log-likelihoods add up).
`binary_nc.py`	The diagnostic on the same two-class task: for nominal and correlation sentences only, decide "correlation" if the mean of $\log p_{\text{corr}} - \log p_{\text{nominal}}$ over the window is positive.
`cross_block.py`	Pools k sentences taken from k *different* batches of the same run (different channel draws) instead of one batch.

Design rationale

"Nested" selection: any choice (which rule, which offset) is made without looking at the fold it's scored on. Otherwise a sweep over many options would look better than it really is, just by chance.

16 The correction experiment in code (`correction_v1`)

16.1 The files

File	Role
`common.py`	loads batch files, models and vocabulary; the LMMSE; complex/real conversion; the 17-number context $\boldsymbol{\kappa}$; decoding; BLEU and token accuracy; recovers the transmitted symbols as $\mathbf{H}^{-1}\times$ faded signal
`restoration_maps.py`	the three map families: `AffineMap` (closed-form ridge with the identity term), `TokenMLP`, `BlockMatrixNet`
`make_dd.py`	the D→D targets for mismatch
`run_fold.py`	one leave-one-seed-out fold: sanity checks, training, every evaluation
`summarize.py`	the tables, as run on the cluster (per-fold ratios averaged)
`summarize_pooled.py`	offline re-summary of the stored folds (pooled gap recovered); the paper's numbers
`launch.sh`	submits the chain D→D → smoke → 5 folds → summary

16.2 What `run_fold.py` does, in order

1. **Sanity checks:** rebuild the stored R_1 from the stored received signal with `common.lmmse` (max error $\leq 2 \times 10^{-6}$), rebuild the faded signal from the recovered symbols, re-decode the stored R_2 (identical sentences 100%). If physics or decoding drifted, the fold would stop here.
2. **Training pool:** the transmitted symbols of the four training seeds.
3. **Physical maps** (`train_physical`): 3,000 steps; each step simulates 4 impaired batches and 1 nominal batch with *fresh* channel and noise draws through the same LMMSE (`simulate`), computes the context on all 128 sentences, the loss on 32 random ones; Adam 10^{-3}, cosine decay.
4. **Mismatch maps:** affine maps fitted in closed form on stored pairs; the R_2 MLP trained 2,000 steps on stored pairs plus 20% D→D identity samples.
5. **Stage 1**, true cause given: every map on every held-out batch; NMSE and cosine at the stage; decode; BLEU-1, BLEU-4, token accuracy.
6. **Variance check:** Spearman correlation between the predicted variance and the actual squared error, per batch.
7. **Nominal harm:** each map on nominal batches.
8. **Wrong-map harm:** each main map on the other causes, `--wrong-map-batches 2` (2 batches per severity).
9. **Routing:** temperature fitted on the other folds' probabilities (grid 0.5 to 6, 45 values, minimum negative log-likelihood); then oracle-cause, hard/soft, block/sentence routing.

16.3 Resources and why

Job	CPUs	Memory	Time limit	Measured
D→D targets	2	4 GB	1 h	completed
Smoke fold (2 batches)	4	8 GB	1 h	completed
Each of 5 folds	4	8 GB	5 h	1 h 10 – 1 h 19, about 2.5 GB
Summary	1	2 GB	20 min	seconds

Design rationale

The limits were sized from a local benchmark with a safety factor, so nothing would time out, without asking for so much that the jobs would wait days in the queue. The smoke fold runs first and the full folds only start if it succeeds (`--dependency=afterok`), so a bug costs one hour, not five.

17 Which features the classical models really use

Note: not recorded by the run

`classical_diagnostics.py` saved scores and confusion matrices, but **not** which 64 features ANOVA or mRMR kept in each fold, nor the elastic net's coefficients (they are only inside the frozen model file). The selection rule is known exactly; the selected lists can be recovered by re-applying the same preprocessing to the feature table, which has not been done.

17.1 What follows directly from the code

- On R_0, ANOVA and mRMR keep **all 51** features ($k = 64 > 51$): no selection happens there.
- On R_1, they keep 64 of 81: 17 dropped.
- On R_2, R_3 and fusion, they keep 64 of 417, 417 and 972 features.
- The elastic net uses all 972 standardized features with an L1 penalty, so some coefficients are exactly zero; how many is in the frozen model.
- The frozen elastic net's classes are stored in **alphabetical** order (scikit-learn sorts labels): imperfect CSI, literal, low SNR, mismatch, normal, OOD, correlation. Reading coefficients in the wrong order would mislabel them, so they must be read through `classes_`.

18 Glossary of the tools

`Pipeline`	scikit-learn chain of steps; `fit` learns every step on training data, `predict` applies them unchanged.
`fit` / `transform`	learn parameters (a median, a mean) / apply them.
NaN	"not a number": a missing value.
`SimpleImputer`	fills missing values (here with the training median).
`StandardScaler`	$z = (x - \mu)/\sigma$ per feature, with training μ, σ.
`SelectKBest(f_classif)`	keeps the k features with the largest ANOVA F.
mRMR	greedy selection: relevance minus redundancy.
Logistic regression	linear scores turned into probabilities by softmax.
`LinearSVC`	linear support vector machine: maximum-margin boundaries.
Elastic net	L1 + L2 penalty: sparse and stable weights.
saga	a stochastic solver that handles the L1 penalty.
Warm start	continue fitting from the previous weights.
Conv1d	a filter sliding along time.
Dilation	spacing between the steps a filter looks at.
Batch normalization	re-centres and re-scales activations per channel.
Masked pooling	mean and max over real (non-padded) steps only.
AdamW	Adam with decoupled weight decay.
Cross-entropy	$-\log$ of the probability given to the true class.
Teacher forcing	feeding the true previous words during training.
Free-running	feeding the model its own previous outputs.
SLURM array	many copies of one job, each with its own index.
`afterok` **dependency**	start only if the previous job succeeded.
SHA-256	a file fingerprint; any change changes it.
`FROZEN` **marker**	file proving a model choice was fixed before arXiv.